

**ST PHILOMENA COLLEGE (AUTONOMOUS)  
PUTTUR  
DEPT OF COMPUTER SCIENCE**



**TITLE OF THE PROJECT**

**“AI Crop Disease Detection and Treatment System”**

**Submitted By:**

**Shruthy. R.S**  
(Reg. no: U05PH23C0198)

**A.V. Ancilla Mary**  
(Reg.no:U05PH23C0201)

**Jeevitha**  
(Reg no: U05PH23C0184)

**Submitted On:27-04-2026**

**Submitted To:**

**Internal Guide**  
**Dr. Vinayachandra**  
Dept. Computer Science  
St. Philomena College (Autonomous)  
Puttur

**Head of Department**  
**Dr. Vinayachandra**  
Dept. Computer Science  
St. Philomena College(Autonomous)  
Puttur

---

# Code Listing and System Interfaces

Python (FastAPI + PyTorch) · React.js · MySQL

## 6.1 Program Code Listing

---

### 6.1.1 Database Connection (database.py)

Establishes the MySQL database connection using SQLAlchemy ORM. The engine connects to the local XAMPP MySQL server at the crop\_disease\_db schema with PyMySQL as the driver.

```
# -----  
# database.py – SQLAlchemy Database Connection  
# This file sets up the connection between the FastAPI backend  
# and the MySQL database running on XAMPP (localhost).  
# -----  
  
# Import create_engine to create the database connection pool  
from sqlalchemy import create_engine  
  
# declarative_base provides the base class for all ORM models  
from sqlalchemy.ext.declarative import declarative_base  
  
# sessionmaker creates a factory for database session objects  
from sqlalchemy.orm import sessionmaker  
  
# — Database URL -----  
# Format: dialect+driver://username:password@host/database_name  
# - mysql+pymysql : use MySQL with PyMySQL driver  
# - root          : default XAMPP MySQL username (no password)  
# - localhost     : MySQL is running on the same machine  
# - crop_disease_db : the database we created in phpMyAdmin  
DATABASE_URL = "mysql+pymysql://root:@localhost/crop_disease_db"  
  
# — Create Engine -----  
# The engine manages the connection pool to the database.  
# It does NOT open a connection immediately – only when needed.  
engine = create_engine(DATABASE_URL)  
  
# — Session Factory -----  
# SessionLocal is a factory that creates individual database  
# sessions for each API request.  
# autocommit=False : we manually commit transactions  
# autoflush=False  : changes are not auto-sent to DB before query  
SessionLocal = sessionmaker(autocommit=False,  
                             autoflush=False, bind=engine)  
  
# — Base Class -----  
# All ORM model classes (User, Crop, Disease, etc.) will inherit  
# from this Base, which tracks table metadata.  
Base = declarative_base()  
  
# — Dependency: get_db() -----
```

```

# This is a FastAPI dependency function used in route handlers.
# It creates a new DB session for each request, and ensures the
# session is properly closed after the request completes –
# even if an exception occurs (via the finally block).
def get_db():
    db = SessionLocal()      # Open a new database session
    try:
        yield db             # Provide the session to the route handler
    finally:
        db.close()          # Always close session after request ends

```

### 6.1.2 Database Models (models.py)

Defines the SQLAlchemy ORM table classes that map directly to MySQL tables: User, Crop, Disease, ScanRecord, and Feedback.

```

# -----
# models.py – SQLAlchemy ORM Table Definitions
# Each class here represents one table in the MySQL database.
# SQLAlchemy automatically maps Python attributes to columns.
# -----

# Import column data types and constraint helpers
from sqlalchemy import (Column, Integer, String, Float,
                        Boolean, DateTime, Text, Enum, ForeignKey)

# func provides SQL functions like NOW() for default timestamps
from sqlalchemy.sql import func

# Import Base from our database connection file
from database import Base

# — Table 1: users —————
# Stores information about farmers who use the application.
class User(Base):
    __tablename__ = "users"          # Actual MySQL table name

    # Primary key – auto-incremented unique ID for each user
    user_id = Column(Integer, primary_key=True, index=True)

    # Farmer's full name – required field (cannot be empty)
    name = Column(String(100), nullable=False)

    # Mobile number – must be unique across all users
    phone = Column(String(15), unique=True)

    # Village or district where the farmer is located
    location = Column(String(100))

    # Automatically records when the account was created
    created_at = Column(DateTime, default=func.now())

```

```

# — Table 2: crops —————
# Master table for crop types supported by the system.
# Currently supports: Arecanut and Tomato.
class Crop(Base):
    __tablename__ = "crops"

    crop_id      = Column(Integer, primary_key=True)

    # Name of the crop (e.g., "Arecanut", "Tomato") – must be unique
    crop_name    = Column(String(50), unique=True, nullable=False)

    # Optional detailed description of the crop
    description  = Column(Text)

# — Table 3: diseases —————
# Stores details about each disease the AI model can detect.
# Linked to the crops table via foreign key.
class Disease(Base):
    __tablename__ = "diseases"

    disease_id   = Column(Integer, primary_key=True)

    # Foreign key linking this disease to its parent crop
    crop_id      = Column(Integer, ForeignKey("crops.crop_id"))

    # Human-readable disease name (e.g., "Bud Rot", "Early Blight")
    disease_name = Column(String(100), nullable=False)

    # The exact class label used by the AI model
    # (e.g., "Arecanut_BudRot") – must be unique
    class_label  = Column(String(100), unique=True)

    # Medical/agricultural description of the disease
    description  = Column(Text)

    # Severity level – restricted to these 4 values only
    severity     = Column(Enum('None', 'Medium', 'High', 'Very High'))

# — Table 4: scan_records —————
# Logs every disease detection performed by the system.
# This forms the detection history for reporting and analysis.
class ScanRecord(Base):
    __tablename__ = "scan_records"

    scan_id      = Column(Integer, primary_key=True)

    # Optional: which user performed the scan (can be anonymous)
    user_id      = Column(Integer, ForeignKey("users.user_id"), nullable=True)

    # Which disease was detected – links to diseases table
    disease_id   = Column(Integer, ForeignKey("diseases.disease_id"))

    # File path of the uploaded leaf image stored on server

```

```

    image_path = Column(String(255), nullable=False)

    # Model confidence score (0.0 to 100.0)
    confidence = Column(Float, nullable=False)

    # Timestamp of when the scan was performed
    scanned_at = Column(DateTime, default=func.now())

# — Table 5: feedback —————
# Stores farmer feedback on whether the detection was correct.
# Used to monitor model performance in the field.
class Feedback(Base):
    __tablename__ = "feedback"

    feedback_id = Column(Integer, primary_key=True)

    # Links this feedback to a specific scan record
    scan_id = Column(Integer, ForeignKey("scan_records.scan_id"))

    # True = model was correct, False = model was wrong
    is_correct = Column(Boolean, nullable=False)

    # Optional written comments from the farmer
    comments = Column(Text)

    # When the feedback was submitted
    submitted_at = Column(DateTime, default=func.now())

```

### 6.1.3 FastAPI Backend (main.py)

The main backend API server. It loads the trained PyTorch MobileNetV2 model on startup and exposes endpoints for disease prediction.

```

# —————
# main.py – FastAPI Backend (PyTorch version)
# This is the core backend server for the crop disease system.
# Run with: uvicorn main:app --reload --host 0.0.0.0 --port 8000
# —————

# FastAPI: modern Python web framework for building APIs
from fastapi import FastAPI, File, UploadFile, HTTPException

# CORSMiddleware: allows the React frontend (port 3000) to call
# this API (port 8000) without browser security blocking it
from fastapi.middleware.cors import CORSMiddleware

# JSONResponse: formats Python dicts as JSON HTTP responses
from fastapi.responses import JSONResponse

# PyTorch: deep learning framework used to run our AI model
import torch
import torch.nn as nn

```

```

# torchvision: provides pre-built models (MobileNetV2) and
# image transformation utilities
from torchvision import models, transforms

# numpy: used for array operations on model output probabilities
import numpy as np

# PIL (Pillow): opens and converts uploaded images
from PIL import Image

# Standard Python libraries for byte streams, JSON, and OS ops
import io, json, os

# — Create FastAPI Application —————
# This creates the main application instance.
# All routes (@app.get, @app.post) are registered on this object.
app = FastAPI(title="Crop Disease Detection API", version="1.0")

# — CORS Middleware —————
# Allow ALL origins, methods, and headers so the React app
# running on http://localhost:3000 can freely call this API.
app.add_middleware(CORSMiddleware, allow_origins=["*"],
                  allow_methods=["*"], allow_headers=["*"])

# — Model Configuration —————
# Total number of disease classes the model was trained on:
# 4 Arecanut classes + 5 Tomato classes = 9
NUM_CLASSES = 9

# Use GPU (CUDA) if available for faster inference,
# otherwise fall back to CPU (most laptops use CPU)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# — load_model() —————
# Builds the MobileNetV2 architecture, replaces the final
# classification layer with our custom 9-class output layer,
# then loads the saved weights from training.
def load_model():
    # Create MobileNetV2 without pre-trained weights
    model = models.mobilenet_v2(weights=None)

    # Replace the default 1000-class output layer with 9 classes
    # model.last_channel = 1280 (MobileNetV2 feature size)
    model.classifier[1] = nn.Linear(model.last_channel, NUM_CLASSES)

    # Load our trained weights saved during train_model.py
    model.load_state_dict(torch.load("model/crop_disease_model.pth",
                                    map_location=device))

    # Move model to CPU/GPU
    model.to(device)

    # Set to evaluation mode: disables dropout and batch norm training
    model.eval()
    return model

```

```

# — Global Variables —————
# These are loaded once at startup and reused for every prediction
MODEL = None # The loaded PyTorch model
LABELS = None # Dict: class_name -> index (from training)
IDX2CLS = None # Reverse dict: index -> class_name

# — Startup Event —————
# This function runs automatically when the server starts.
# It loads the model and labels if they exist on disk.
@app.on_event("startup")
def startup_event():
    global MODEL, LABELS, IDX2CLS
    # Only load if both required files are present
    if os.path.exists("model/crop_disease_model.pth") and
os.path.exists("model/class_labels.json"):
        MODEL = load_model()
        # Load the class label mapping saved during training
        LABELS = json.load(open("model/class_labels.json"))
        # Invert it: {0: "Arecanut_BudRot", 1: "Arecanut_Healthy", ...}
        IDX2CLS = {v: k for k, v in LABELS.items()}

# — Treatment Database —————
# A dictionary mapping each class label to its disease info.
# This is returned alongside the AI prediction to give farmers
# actionable treatment and prevention guidance.
TREATMENTS = {
    "Arecanut_Healthy": {
        "status": "Healthy",
        "description": "Plant is healthy. No disease detected.",
        "treatment": ["Continue regular watering and fertilization.",
            "Apply balanced NPK fertilizer every 3 months.",
            "Ensure proper drainage to prevent waterlogging."],
        "severity": "None",
        "prevention": ["Regular monitoring", "Maintain field hygiene"]
    },
    "Arecanut_BudRot": {
        "status": "Diseased",
        "description": "Bud Rot caused by Phytophthora palmivora fungus.",
        "treatment": ["Remove and destroy infected plant parts immediately.",
            "Apply Bordeaux mixture (1%) to the crown of the palm.",
            "Spray Metalaxyl + Mancozeb @ 2g/L water.",
            "Avoid overhead irrigation to reduce humidity."],
        "severity": "High",
        "prevention": ["Good drainage", "Avoid bud injuries"]
    },
    "Arecanut_Koleroga": {
        "status": "Diseased",
        "description": "Koleroga (Fruit Rot) caused by Phytophthora meadii.",
        "treatment": ["Spray Bordeaux mixture (1%) before monsoon.",
            "Apply Copper oxychloride @ 3g/L on bunches.",
            "Remove and bury all fallen fruits away from field."],
        "severity": "High",
        "prevention": ["Pre-monsoon spraying", "Remove fallen fruits"]
    },
}

```

```

"Arecanut_YellowLeaf": {
  "status": "Diseased",
  "description": "Yellow Leaf Disease caused by phytoplasma.",
  "treatment": ["Remove and destroy severely infected palms.",
    "Spray Imidacloprid @ 0.5ml/L to control leafhoppers.",
    "Apply micronutrient mixture (zinc, boron)."],
  "severity": "High",
  "prevention": ["Control leafhoppers", "Use certified seedlings"]
},
"Tomato_Healthy": {
  "status": "Healthy",
  "description": "Plant is healthy. No disease detected.",
  "treatment": ["Continue regular watering schedule.",
    "Apply balanced fertilizer weekly."],
  "severity": "None",
  "prevention": ["Crop rotation", "Proper spacing"]
},
"Tomato_EarlyBlight": {
  "status": "Diseased",
  "description": "Early Blight caused by Alternaria solani fungus.",
  "treatment": ["Remove infected lower leaves immediately.",
    "Spray Mancozeb @ 2.5g/L water every 7-10 days.",
    "Apply Chlorothalonil @ 2g/L water."],
  "severity": "Medium",
  "prevention": ["Crop rotation", "Mulching"]
},
"Tomato_LateBlight": {
  "status": "Diseased",
  "description": "Late Blight caused by Phytophthora infestans.",
  "treatment": ["Act immediately – this disease spreads very fast.",
    "Spray Metalaxyl + Mancozeb @ 2.5g/L water.",
    "Repeat every 5-7 days during cool wet weather."],
  "severity": "Very High",
  "prevention": ["Resistant varieties", "Avoid overhead irrigation"]
},
"Tomato_LeafCurl": {
  "status": "Diseased",
  "description": "Leaf Curl Virus spread by whiteflies.",
  "treatment": ["Spray Imidacloprid @ 0.5ml/L to control whiteflies.",
    "Use yellow sticky traps to catch whiteflies.",
    "Remove and destroy heavily infected plants."],
  "severity": "High",
  "prevention": ["Insect-proof nets", "Reflective mulches"]
},
"Tomato_SeptoriaLeafSpot": {
  "status": "Diseased",
  "description": "Septoria Leaf Spot caused by Septoria lycopersici.",
  "treatment": ["Remove infected leaves immediately.",
    "Spray Chlorothalonil @ 2g/L water every 7-10 days.",
    "Apply Copper oxychloride @ 3g/L water."],
  "severity": "Medium",
  "prevention": ["Crop rotation", "Remove plant debris"]
}
}

```

```

# — Image Preprocessing Pipeline —————
# Defines the sequence of transformations applied to every
# uploaded image before feeding it into the model.
# Must exactly match the transforms used during training.
preprocess_transform = transforms.Compose([
    # Step 1: Resize image to 224x224 pixels (MobileNetV2 input size)
    transforms.Resize((224, 224)),

    # Step 2: Convert PIL image to PyTorch tensor (values 0.0-1.0)
    transforms.ToTensor(),

    # Step 3: Normalize using ImageNet mean and std values.
    # This standardizes pixel values so the pre-trained weights
    # work correctly on our custom dataset.
    transforms.Normalize([0.485, 0.456, 0.406],    # Mean (R, G, B)
                          [0.229, 0.224, 0.225])  # Std  (R, G, B)
])

def preprocess(img_bytes: bytes):
    # Open raw bytes as a PIL Image and ensure RGB (3 channels)
    img = Image.open(io.BytesIO(img_bytes)).convert("RGB")

    # Apply all transforms defined above
    tensor = preprocess_transform(img)

    # Add batch dimension: shape becomes [1, 3, 224, 224]
    # (model expects a batch, even for single images)
    tensor = tensor.unsqueeze(0)

    # Move tensor to the same device as the model (CPU or GPU)
    return tensor.to(device)

# — Route: GET / —————
# Health check endpoint – confirms the API server is running.
# Visit http://localhost:8000/ in a browser to verify.
@app.get("/")
def root():
    return {"message": "Crop Disease Detection API is running"}

# — Route: POST /predict —————
# Main prediction endpoint.
# Accepts a leaf image file, runs it through the AI model,
# and returns the disease name, confidence, and treatment info.
@app.post("/predict")
async def predict(file: UploadFile = File(...)):

    # Validate that the uploaded file is an image (JPG or PNG)
    if file.content_type not in ["image/jpeg", "image/png", "image/jpg"]:
        raise HTTPException(400, "Only JPG/PNG images are accepted.")

    # Check that the model was successfully loaded at startup
    if MODEL is None:
        raise HTTPException(503, "Model not loaded. Please train first.")

    # Read the raw bytes of the uploaded image file

```

```

img_bytes = await file.read()

# Apply preprocessing transforms to prepare for model input
inp = preprocess(img_bytes)

# Run inference – torch.no_grad() disables gradient tracking
# (not needed for prediction, saves memory and speeds up)
with torch.no_grad():
    outputs = MODEL(inp)  # Raw scores (logits) for 9 classes

    # Convert logits to probabilities using Softmax
    # All 9 values now sum to 1.0 (100%)
    probs = torch.softmax(outputs, dim=1)[0].cpu().numpy()

# Find the class with the highest probability
idx = int(np.argmax(probs))  # Index of predicted class
conf = float(np.max(probs)) * 100  # Confidence as percentage
cls = IDX2CLS[idx]  # Class name (e.g.,
"Tomato_LateBlight")

# Look up treatment info for this predicted disease
info = TREATMENTS.get(cls, {})

# Build Top-3 predictions sorted by confidence (highest first)
top3 = [{"disease": IDX2CLS[i],
        "confidence": round(float(probs[i]) * 100, 2)}
        for i in np.argsort(probs)[::-1][:3]]

# Return structured JSON response to the React frontend
return JsonResponse({
    "success": True,
    "predicted_class": cls,  # Full label
    "confidence": round(conf, 2),  # e.g., 83.91
    "crop": "Arecanut" if "Arecanut" in cls else "Tomato",
    "disease_name": cls.split("_", 1)[1],  # e.g.,
"LateBlight"
    "status": info.get("status", "Unknown"),
    "description": info.get("description", ""),
    "severity": info.get("severity", "Unknown"),
    "treatment": info.get("treatment", []),
    "prevention": info.get("prevention", []),
    "top3_predictions": top3
})

```

### 6.1.4 Model Training (train\_model.py)

Trains the MobileNetV2 CNN using PyTorch transfer learning in two phases. Achieved 90.28% validation accuracy and 89.00% test accuracy.

```

# -----
# train_model.py – CNN Model Training (PyTorch version)
# Trains a MobileNetV2 model on the crop disease dataset using
# Transfer Learning in two phases.

```

```

# Run: python train_model.py (inside activated venv)
# -----

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, models, transforms
import json, os

# — Training Configuration —————
IMG_SIZE    = (224, 224)          # Input size required by MobileNetV2
BATCH_SIZE  = 32                  # Number of images processed together
EPOCHS      = 30                  # Total training epochs (15+15)
DATASET_DIR = "../dataset/"      # Path to train/val/test folders
MODEL_PATH  = "model/crop_disease_model.pth"  # Where to save model
LABELS_PATH = "model/class_labels.json"      # Where to save labels

# — Device Setup —————
# Automatically use GPU if available, else use CPU.
# Training on CPU is slower but works without a graphics card.
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# — Training Transforms (Data Augmentation) —————
# Augmentation randomly modifies training images to make the
# model more robust and prevent overfitting.
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),          # Resize to model input size
    transforms.RandomHorizontalFlip(),      # Randomly flip left-right
    transforms.RandomRotation(30),          # Rotate up to 30 degrees
    transforms.ColorJitter(                 # Random brightness/contrast
        brightness=0.2, contrast=0.2),
    transforms.ToTensor(),                  # Convert image to tensor
    transforms.Normalize(                   # Normalize with ImageNet stats
        [0.485, 0.456, 0.406],
        [0.229, 0.224, 0.225])
])

# — Validation/Test Transforms (No Augmentation) —————
# Validation images are NOT augmented – we want consistent
# measurements of model performance.
val_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
        [0.229, 0.224, 0.225])
])

# — Load Datasets —————
# ImageFolder automatically reads subfolders as class labels.
# Folder structure: dataset/train/Arecanut_BudRot/img1.jpg ...
train_dataset = datasets.ImageFolder(
    os.path.join(DATASET_DIR, 'train'), transform=train_transform)
val_dataset   = datasets.ImageFolder(

```

```

    os.path.join(DATASET_DIR, 'val'), transform=val_transform)
test_dataset = datasets.ImageFolder(
    os.path.join(DATASET_DIR, 'test'), transform=val_transform)

# — Create Data Loaders —————
# DataLoader batches images and handles parallel loading.
# shuffle=True for training keeps the model from memorizing order.
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
                           shuffle=True, num_workers=0)
val_loader   = DataLoader(val_dataset, batch_size=BATCH_SIZE,
                           shuffle=False, num_workers=0)
test_loader  = DataLoader(test_dataset, batch_size=BATCH_SIZE,
                           shuffle=False, num_workers=0)

# — Save Class Labels —————
# Save the class-to-index mapping so main.py can decode
# model output indices back to disease names at inference time.
NUM_CLASSES = len(train_dataset.classes)
print(f"Classes ({NUM_CLASSES}): {train_dataset.classes}")
os.makedirs("model", exist_ok=True)
with open(LABELS_PATH, 'w') as f:
    json.dump(train_dataset.class_to_idx, f)

# — Build Model —————
# Load MobileNetV2 pre-trained on ImageNet (1.2M images, 1000 classes).
# Transfer learning reuses these learned visual features for our task.
model = models.mobilenet_v2(weights='IMAGENET1K_V1')

# Freeze all layers – we don't want to change pre-trained weights yet
for param in model.parameters():
    param.requires_grad = False

# Replace the final classification layer:
# Original: 1000 ImageNet classes → Our: 9 disease classes
# model.last_channel = 1280 (MobileNetV2 output feature size)
model.classifier[1] = nn.Linear(model.last_channel, NUM_CLASSES)
model = model.to(device) # Move model to CPU/GPU

# — Loss Function & Optimizer —————
# CrossEntropyLoss: standard loss for multi-class classification
criterion = nn.CrossEntropyLoss()

# Adam optimizer with learning rate 0.001 for Phase 1
# Only optimize classifier parameters (base is frozen)
optimizer = optim.Adam(model.classifier.parameters(), lr=0.001)

# Reduce learning rate when validation loss stops improving
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.2, patience=3)

# — Training Function —————
# Runs one complete pass through the training data.
# Returns average loss and accuracy for that epoch.
def train_epoch(loader):
    model.train() # Enable dropout and batch norm training mode

```

```

total_loss, correct, total = 0, 0, 0
for imgs, labels in loader:
    imgs, labels = imgs.to(device), labels.to(device)

    optimizer.zero_grad()          # Clear gradients from last step
    outputs = model(imgs)           # Forward pass through model
    loss = criterion(outputs, labels) # Compute prediction error
    loss.backward()                 # Backpropagate gradients
    optimizer.step()                # Update model weights

    total_loss += loss.item()
    correct    += (outputs.argmax(1) == labels).sum().item()
    total      += labels.size(0)

return total_loss / len(loader), correct / total

# — Evaluation Function —————
# Runs one pass through validation/test data without updating weights.
def eval_epoch(loader):
    model.eval() # Disable dropout, use running batch norm stats
    total_loss, correct, total = 0, 0, 0
    with torch.no_grad(): # Disable gradient tracking (faster)
        for imgs, labels in loader:
            imgs, labels = imgs.to(device), labels.to(device)
            outputs = model(imgs)
            loss = criterion(outputs, labels)
            total_loss += loss.item()
            correct    += (outputs.argmax(1) == labels).sum().item()
            total      += labels.size(0)
    return total_loss / len(loader), correct / total

# — Phase 1: Train Classifier Only (Epochs 1-15) —————
# Only the new classification head is trained here.
# The pre-trained MobileNetV2 base remains completely frozen.
print("\n=== Phase 1: Training classifier layers ===")
best_val_acc = 0
for epoch in range(15):
    train_loss, train_acc = train_epoch(train_loader)
    val_loss, val_acc = eval_epoch(val_loader)
    scheduler.step(val_loss) # Adjust LR if loss plateaus

    print(f"Epoch {epoch+1}/15 | "
          f"Train Loss: {train_loss:.4f} Acc: {train_acc*100:.2f}% | "
          f"Val Loss: {val_loss:.4f} Acc: {val_acc*100:.2f}%")

    # Save model only when validation accuracy improves
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), MODEL_PATH)
        print(f" Model saved (val_acc={val_acc*100:.2f}%)")

# — Phase 2: Fine-tune Last 30 Layers (Epochs 16-30) —————
# Unfreeze the entire model, then re-freeze all except last 30 layers.
# Use a much smaller learning rate (0.0001) to avoid overwriting
# the valuable pre-trained features learned on ImageNet.

```

```

print("\n=== Phase 2: Fine-tuning ===")
for param in model.parameters():
    param.requires_grad = True      # Unfreeze all layers

all_params = list(model.parameters())
for param in all_params[:-30]:      # Re-freeze everything except last 30
    param.requires_grad = False

# New optimizer with lower learning rate for fine-tuning
optimizer2 = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()), lr=0.0001)
scheduler2 = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer2, mode='min', factor=0.2, patience=3)

for epoch in range(15, EPOCHS):
    train_loss, train_acc = train_epoch(train_loader)
    val_loss, val_acc = eval_epoch(val_loader)
    scheduler2.step(val_loss)

    print(f"Epoch {epoch+1}/{EPOCHS} | "
          f"Train Loss: {train_loss:.4f} Acc: {train_acc*100:.2f}% | "
          f"Val Loss: {val_loss:.4f} Acc: {val_acc*100:.2f}%")

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), MODEL_PATH)
        print(f" Model saved (val_acc={val_acc*100:.2f}%)")

# — Final Evaluation on Test Set —————
# Load the best saved model and evaluate on unseen test data.
# This gives the true real-world accuracy of the system.
print("\n=== Test Set Evaluation ===")
model.load_state_dict(torch.load(MODEL_PATH, map_location=device))
test_loss, test_acc = eval_epoch(test_loader)
print(f"Test Accuracy: {test_acc*100:.2f}% | Loss: {test_loss:.4f}")
print(f"Model saved at: {MODEL_PATH}")
# Result: Test Accuracy: 89.00% | Validation Accuracy: 90.28%

```

### 6.1.5 React Frontend (App.jsx)

The single-page React application providing the farmer-facing UI. Handles image upload, API communication, and result display.

```

// —————
// App.jsx – React Frontend (Main Component)
// This is the complete single-page UI for the crop disease app.
// It handles image upload, sends it to the FastAPI backend,
// and displays the AI prediction with treatment information.
// —————

// useState: manages component state (image, result, loading, etc.)
// useRef: creates a reference to the hidden file input element
import { useState, useRef } from "react";

```

```

// Base URL of our FastAPI backend server
const API = "http://localhost:8000";

// Color mapping for severity badges shown in the result card
// Green = healthy/none, Yellow = medium, Red = high, Dark red = very high
const severityColor = {
  "None":      "#22c55e",    // Green
  "Medium":    "#f59e0b",    // Orange/Yellow
  "High":      "#ef4444",    // Red
  "Very High": "#7f1d1d"     // Dark Red
};

// — Main App Component —————
export default function App() {

  // img: the raw File object selected by the user
  const [img, setImg] = useState(null);

  // preview: object URL used to display the selected image in UI
  const [preview, setPreview] = useState(null);

  // result: the JSON response from the /predict API endpoint
  const [result, setResult] = useState(null);

  // loading: true while waiting for API response (shows "Analyzing...")
  const [loading, setLoading] = useState(false);

  // error: stores any error message to display to the user
  const [error, setError] = useState(null);

  // fileRef: direct reference to the hidden <input type="file"> element
  // Used to trigger the file picker from Gallery and Camera buttons
  const fileRef = useRef();

  // — handleFile() —————
  // Called when user selects a file via input or drag-and-drop.
  // Saves the file and creates a preview URL.
  const handleFile = (f) => {
    if (!f) return;
    setImg(f); // Save File object for upload
    setPreview(URL.createObjectURL(f)); // Create preview URL for display
    setResult(null); // Clear previous result
    setError(null); // Clear previous errors
  };

  // — handleDrop() —————
  // Handles drag-and-drop image uploads onto the upload box.
  const handleDrop = (e) => {
    e.preventDefault(); // Prevent browser from opening
    file // Get the dropped file
    handleFile(e.dataTransfer.files[0]);
  };

  // — detect() —————

```

```

// Sends the selected image to the FastAPI /predict endpoint.
// Uses FormData to send the file as multipart/form-data.
const detect = async () => {
  if (!img) return;          // Do nothing if no image selected
  setLoading(true);          // Show "Analyzing..." on button
  setError(null);            // Clear previous errors

  try {
    // FormData allows sending files in HTTP requests
    const fd = new FormData();
    fd.append("file", img);   // Key must match FastAPI parameter name

    // POST the image to the backend prediction endpoint
    const res = await fetch(API + "/predict",
                             { method: "POST", body: fd });

    // If server returned error status, throw to catch block
    if (!res.ok) throw new Error("Server error. Check backend.");

    // Parse the JSON response and save to state for display
    setResult(await res.json());

  } catch (e) {
    // Display the error message to the user
    setError(e.message);
  } finally {
    // Always turn off loading spinner, even if error occurred
    setLoading(false);
  }
};

```

```

// — Render (UI) —————
return (
  // Main container: dark green background, full screen height
  <div style={{ minHeight:"100vh", background:"#064e3b",
               padding:"20px", fontFamily:"Segoe UI,sans-serif" }}>

    {/* — Header Section — */}
    <div style={{ textAlign:"center", color:"#fff", marginBottom:24 }}>
      <h1>Crop Disease Detector</h1>
      <p>Arecanut and Tomato – AI Powered Detection</p>
    </div>

    {/* — Upload Card — */}
    <div style={{ background:"#fff", borderRadius:16,
               padding:24, maxWidth:480, margin:"0 auto" }}>

      {/* Image Drop Zone: click or drag-and-drop to upload */}
      <div
        onDrop={handleDrop}
        onDragOver={e => e.preventDefault()} // Allow drop
        onClick={() => fileRef.current.click()} // Open file picker
        style={{ border:"2px dashed #6ee7b7", borderRadius:12,
                  cursor:"pointer", textAlign:"center",
                  background:"#f0fdf4" }}>

```

```

    /* Show image preview if selected, else show prompt */
    {preview
      ? <img src={preview} alt="leaf"
        style={{ width:"100%", maxHeight:240,
          objectFit:"contain" }} />
      : <p>Tap to upload or drag and drop leaf image</p>
    }
  </div>

  /* Hidden file input: triggered by Gallery/Camera buttons */
  /* accept="image/*" : only image files allowed */
  /* capture="environment" : prefers rear camera on mobile */
  <input ref={fileRef} type="file"
    accept="image/*" capture="environment"
    style={{ display:"none" }}
    onChange={e => handleFile(e.target.files[0])} />

  /* Gallery and Camera buttons – both open the file picker */
  <div style={{ display:"flex", gap:10, marginTop:12 }}>
    <button onClick={() => fileRef.current.click()}>
      Gallery
    </button>
    <button onClick={() => fileRef.current.click()}>
      Camera
    </button>
  </div>

  /* Detect Disease button – disabled until image is selected */
  <button onClick={detect} disabled={!img || loading}
    style={{ width:"100%", marginTop:12,
      background: img ? "#059669" : "#d1d5db" }}>
    /* Show "Analyzing..." while waiting for API response */
    {loading ? "Analyzing..." : "Detect Disease"}
  </button>

  /* Display error message if API call fails */
  {error && <p style={{ color:"red" }}>{error}</p>}
</div>

/* — Result Card (only shown after successful prediction) — */
{result && (
  <div style={{ background:"#fff", borderRadius:16, padding:24,
    maxWidth:480, margin:"18px auto" }}>

    /* Disease name (e.g., "SeptoriaLeafSpot") */
    <h2>{result.disease_name}</h2>

    /* Crop type and confidence percentage */
    <p>Crop: {result.crop} | Confidence: {result.confidence}%</p>

    /* Severity badge with color coding */
    <span style={{ background: severityColor[result.severity],
      color:"#fff", padding:"3px 10px",
      borderRadius:8, fontSize:13 }}>
      Severity: {result.severity}
  </div>
)}

```

```

</span>

    {/* Disease description from TREATMENTS dictionary */}
    <p style={{ marginTop:12 }}>{result.description}</p>

    {/* Numbered list of treatment steps */}
    <h3>Treatment Steps</h3>
    <ol>
      {result.treatment.map((t, i) => <li key={i}>{t}</li>)}
    </ol>

    {/* Bulleted list of prevention tips */}
    <h3>Prevention</h3>
    <ul>
      {result.prevention.map((p, i) => <li key={i}>{p}</li>)}
    </ul>
  </div>
)}
</div>
);
}
// End of App.jsx

```

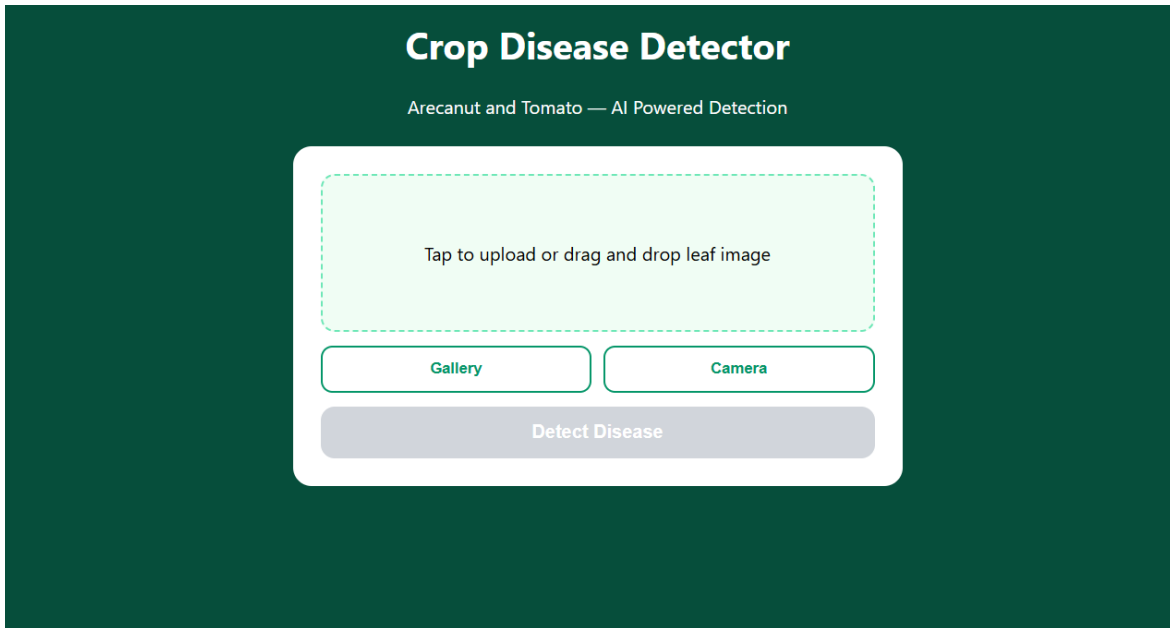
## 6.2 System Interfaces

---

The following screenshots illustrate the working interfaces of the AI Crop Disease Detection and Treatment System.

### 6.2.1 Home Screen — Upload Interface

The main landing screen allows farmers to upload a leaf image by tapping the image area, using the Gallery button, or Camera button. The dark green theme conveys an agricultural context. The Detect Disease button activates once an image is selected.



### 6.2.2 Detection Result — Tomato Septoria Leaf Spot

The AI model detects Tomato Septoria Leaf Spot with 83.91% confidence. Severity is shown as Medium with full treatment steps and prevention.

# Crop Disease Detector

Arecanut and Tomato — AI Powered Detection



Gallery

Camera

Detect Disease

## SeptoriaLeafSpot

Crop: Tomato | Confidence: 83.91%

# SeptoriaLeafSpot

Crop: Tomato | Confidence: 83.91%

Severity: Medium

Septoria Leaf Spot caused by *Septoria lycopersici*.

## Treatment Steps

1. Remove infected leaves immediately.
2. Spray Chlorothalonil @ 2g/L water every 7-10 days.
3. Apply Copper oxychloride @ 3g/L water.

## Prevention

- Crop rotation
- Remove plant debris

### 6.2.3 Detection Result — Arecanut Koleroga

Koleroga (Fruit Rot) detected with 99.99% confidence — demonstrating the model's high accuracy on Arecanut diseases.

## Crop Disease Detector

Arecanut and Tomato — AI Powered Detection



Gallery

Camera

Detect Disease

### Koleroga

Crop: Arecanut | Confidence: 99.99%

# Koleroga

Crop: Arecanut | Confidence: 99.99%

Severity: High

Koleroga (Fruit Rot) caused by *Phytophthora meadii*.

## Treatment Steps

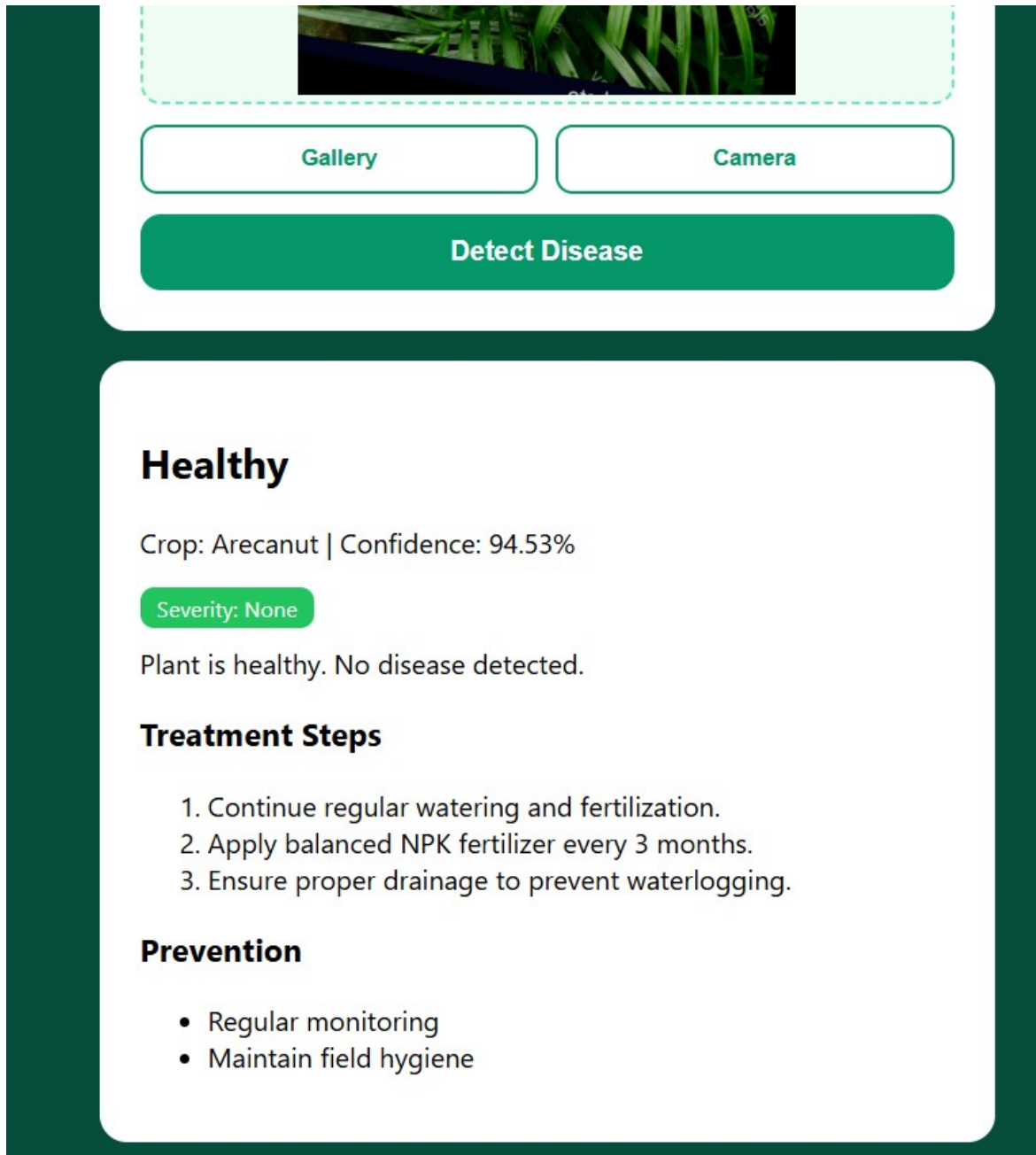
1. Spray Bordeaux mixture (1%) before monsoon.
2. Apply Copper oxychloride @ 3g/L on bunches.
3. Remove and bury all fallen fruits away from field.

## Prevention

- Pre-monsoon spraying
- Remove fallen fruits

### 6.2.4 Detection Result — Arecanut Healthy

When a healthy plant is scanned, the system correctly reports Severity: None and provides standard care recommendation



### 6.2.5 System Architecture Summary

| Component      | Technology / Details                      |
|----------------|-------------------------------------------|
| AI Model       | MobileNetV2 (Transfer Learning) — PyTorch |
| Training Data  | 9 classes: 4 Arecanut + 5 Tomato diseases |
| Model Accuracy | 90.28% Validation   89.00% Test Accuracy  |
| Backend        | FastAPI + Uvicorn (Python 3.12)           |
| Database       | MySQL via XAMPP + SQLAlchemy ORM          |
| Frontend       | React.js (JSX) — Responsive Web UI        |
| Image Input    | JPG / PNG via Gallery or Camera           |
| API Endpoint   | POST /predict — returns JSON prediction   |